

JAVA INTERVIEW PREPARATION

CHAPTER 75

CREATIONAL PATTERNS AND
DEPENDENCY CONSTRUCTION

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Creational Patterns and Dependency Construction in Modern Java

Senior / Lead Java Developer Reading Chapter

Creational patterns control how objects come into existence, which implementation is selected, and who owns the resulting lifecycle. Their purpose is not to hide every constructor. They are useful when construction contains policy: validation, family compatibility, expensive setup, environment selection, optional steps, or a lifecycle boundary.

Senior interviews often ask for Factory, Builder, Singleton, Prototype, and dependency injection as definitions. A stronger answer begins with the design pressure, shows why ordinary construction has become unstable, and explains the cost introduced by the pattern. Modern Java features such as records, lambdas, sealed types, [ServiceLoader](#), and dependency-injection containers change how these patterns are expressed but do not remove the underlying responsibilities.

Patterns are named trade-offs

A pattern is a reusable arrangement of responsibilities, not a mandatory class diagram. Use it when the same design force appears repeatedly. Avoid adding [Factory](#), [Builder](#), or [Manager](#) merely to make a codebase look architectural.

```
direct construction is sufficient

Order order = new Order(id, lines, customer);

construction policy is changing

caller -> construction boundary -> selected implementation
                                     -> validation and defaults
                                     -> lifecycle ownership
```

Every additional abstraction increases navigation, testing surface, and configuration. The pattern should reduce a more important coupling than it creates.

Constructor first

Start with the simplest correct design. A constructor is explicit, compile-time checked, and easy to discover.

```
public final class Money {
    private final BigDecimal amount;
    private final Currency currency;

    public Money(BigDecimal amount, Currency currency) {
        this.amount = Objects.requireNonNull(amount);
        this.currency = Objects.requireNonNull(currency);
        if (amount.scale() > currency.getDefaultFractionDigits()) {
            throw new IllegalArgumentException("unsupported scale");
        }
    }
}
```

Do not replace a clear constructor with a pattern unless construction pressure exists. Static factories can improve names, caching, subtype selection, and validation while remaining compact:

```
Money zero = Money.zero(usd);
Money parsed = Money.parse("12.50 USD");
```

A static factory is not the Gang-of-Four Factory Method pattern by itself. It is a useful Java creation technique. Precise terminology helps, but design quality matters more than arguing over labels.

Simple factory centralizes selection

Suppose a checkout workflow selects a payment connector from configuration.

```
interface PaymentGateway {
    PaymentResult charge(PaymentRequest request);
}

final class PaymentGatewayFactory {
    static PaymentGateway create(GatewayType type,
                                GatewayConfig config,
                                HttpClient client) {
        return switch (type) {
            case STRIPE -> new StripeGateway(config.stripe(), client);
            case ADYEN -> new AdyenGateway(config.adyen(), client);
        };
    }
}
```

The factory moves implementation selection out of business code. The caller depends on `PaymentGateway` and no longer contains repeated switches. This is useful even though it is often called a simple factory rather than the formal Factory Method pattern.

The factory now owns a policy boundary. It must validate configuration, define unsupported cases, and decide whether gateways are new per call or shared. If it opens resources, lifecycle ownership must be clear. Returning a new HTTP connection pool for every payment would be disastrous even though the selection code is correct.

Factory Method delegates one creation step

Factory Method defines an operation for creating a collaborator and lets a subtype or supplied strategy choose the concrete product while the surrounding workflow remains fixed.

```
abstract class ImportJob {
    public final ImportReport run(Path input) {
        RecordReader reader = createReader(input);
        try (reader) {
            return importAll(reader);
        }
    }

    protected abstract RecordReader createReader(Path input);
}

final class CsvImportJob extends ImportJob {
    @Override
    protected RecordReader createReader(Path input) {
        return new CsvRecordReader(input);
    }
}
```

fixed workflow in creator

```
run()
|
+--> createReader() <--- variable construction step
|
+--> process, close, report
```

This combines Factory Method with Template Method. Inheritance is justified only if the workflow is a genuine stable extension point. Composition is often more flexible:

```
record ImportJob(Function<Path, RecordReader> readerFactory) {
    ImportReport run(Path input) {
        try (RecordReader reader = readerFactory.apply(input)) {
            return importAll(reader);
        }
    }
}
```

A lambda can therefore carry factory behavior without a hierarchy. It does not eliminate the pattern; it changes its implementation form.

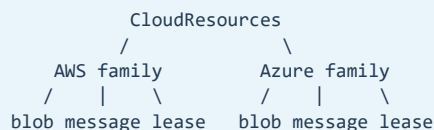
Abstract Factory creates compatible families

Abstract Factory is useful when several related products must be selected together. The critical idea is family compatibility, not merely several factory methods.

```
interface CloudResources {
    BlobStore blobStore();
    MessagePublisher publisher();
    LeaseStore leaseStore();
}

final class AwsResources implements CloudResources {
    // returns S3, SNS/SQS, and DynamoDB-based collaborators
}

final class AzureResources implements CloudResources {
    // returns Blob Storage, Service Bus, and Azure lease collaborators
}
```



one environment selects one coherent family

The application receives one family boundary instead of independently choosing storage, messaging, and locking. Independent selection could create unsupported combinations or inconsistent credentials and observability.

Abstract Factory becomes awkward when every consumer needs a different subset or when the family interface grows with unrelated products. Split factories by cohesive capability or use dependency injection with validated configuration. A factory with 40 methods is a service locator disguised as a pattern.

Registration replaces growing switches

An extension-oriented factory can register providers by a stable key.

```
final class CodecRegistry {
    private final Map<String, Supplier<Codec>> providers;

    CodecRegistry(Map<String, Supplier<Codec>> providers) {
        this.providers = Map.copyOf(providers);
    }

    Codec create(String name) {
        Supplier<Codec> provider = providers.get(name);
        if (provider == null) throw new IllegalArgumentException("codec: " + name);
        return provider.get();
    }
}
```

Registration supports plugins and tests without editing a central switch. It also introduces duplicate-key policy, initialization order, trusted-provider concerns, and lifecycle questions. A [Supplier](#) implies new or provider-controlled creation; do not assume singleton behavior.

Java's [ServiceLoader](#) can discover provider implementations declared through the module system or service metadata. Discovery is not dependency injection by itself. Providers still need configuration, error isolation, version compatibility, and security review.

Builder makes staged construction readable

Builder is valuable when an object has many optional parameters, construction requires named steps, or validation spans several fields.

```

public final class HttpRequestPolicy {
    private final Duration connectTimeout;
    private final Duration requestTimeout;
    private final int maxAttempts;
    private final Set<Integer> retryableStatuses;

    private HttpRequestPolicy(Builder builder) {
        connectTimeout = builder.connectTimeout;
        requestTimeout = builder.requestTimeout;
        maxAttempts = builder.maxAttempts;
        retryableStatuses = Set.copyOf(builder.retryableStatuses);
        validate();
    }

    public static Builder builder() { return new Builder(); }

    public static final class Builder {
        private Duration connectTimeout = Duration.ofSeconds(2);
        private Duration requestTimeout = Duration.ofSeconds(5);
        private int maxAttempts = 1;
        private Set<Integer> retryableStatuses = Set.of(502, 503, 504);

        public Builder connectTimeout(Duration value) {
            connectTimeout = Objects.requireNonNull(value);
            return this;
        }

        public Builder requestTimeout(Duration value) {
            requestTimeout = Objects.requireNonNull(value);
            return this;
        }

        public Builder maxAttempts(int value) {
            maxAttempts = value;
            return this;
        }

        public HttpRequestPolicy build() {
            return new HttpRequestPolicy(this);
        }
    }
}

```

The built object copies mutable collections and validates the complete invariant. The builder may remain mutable and should normally be request-local. Reusing it concurrently or after a build can create surprising results unless explicitly supported.

Builder does not make invalid intermediate states impossible; it delays validation until `build`. A step builder can enforce call order through interfaces, but often creates many types and poor usability. Required constructor parameters plus a builder for optional values is a practical compromise.

```

required identity -> constructor/static factory
optional policy  -> builder methods
build()          -> complete validation + defensive copy

```

Records already provide concise transparent data construction. A builder around a three-field record may add ceremony without value. Use one when naming, defaults, compatibility evolution, or cross-field validation justifies it.

Builder and API evolution

Adding an optional builder method can preserve source compatibility better than adding a constructor parameter. However, default changes are behavioral changes. If persisted configuration omits a field, upgrading may silently change behavior.

Version defaults explicitly for operationally sensitive policies. Log the resolved configuration, not secrets, and test old configuration files against new builders. Generated builders can reduce boilerplate but do not decide validation or ownership for you.

Singleton controls instance count and access

Singleton combines two ideas: one instance under a scope and global access to it. The first can be useful; the second often creates hidden dependencies.

The safest basic Java singleton uses an enum:

```
enum MetricsRegistry {  
    INSTANCE;  
  
    void record(String name, long value) { }  
}
```

Enum construction is class-loader-safe and serialization-safe under ordinary JVM rules. But "one" means one per defining class loader, not necessarily one per machine, cluster, tenant, or test process.

The initialization-on-demand holder idiom is also lazy:

```
final class ClockProvider {  
    private ClockProvider() { }  
  
    private static class Holder {  
        static final ClockProvider INSTANCE = new ClockProvider();  
    }  
  
    static ClockProvider instance() { return Holder.INSTANCE; }  
}
```

Double-checked locking requires a `volatile` field because construction and publication must be safely ordered:

```
private static volatile ExpensiveClient instance;  
  
static ExpensiveClient instance() {  
    ExpensiveClient local = instance;  
    if (local == null) {  
        synchronized (ExpensiveClient.class) {  
            local = instance;  
            if (local == null) instance = local = new ExpensiveClient();  
        }  
    }  
    return local;  
}
```

Use simpler JVM initialization unless delayed construction truly requires this complexity.

Why global Singleton causes pain

Global calls hide dependencies:

```
class PricingService {  
    Money price(Item item) {  
        return ExchangeRates.INSTANCE.convert(...);  
    }  
}
```

Tests cannot easily substitute rates. Lifecycle and configuration are pulled from ambient state. Parallel tests can interfere, and shutdown ownership is unclear. Prefer injecting one application-scoped instance:

```
final class PricingService {
    private final ExchangeRates rates;

    PricingService(ExchangeRates rates) {
        this.rates = rates;
    }
}
```

The dependency-injection container may still create only one instance, but consumers depend explicitly on the abstraction. "Singleton scope" and "Singleton pattern with global accessor" are not the same design.

Singleton scope is not distributed uniqueness

A Spring singleton is one bean instance per application context. Multiple contexts, JVMs, pods, or regions each have their own instance. It cannot enforce one scheduler leader or one sequence generator across a cluster.

Distributed uniqueness requires coordination such as a database constraint, lease, consensus-backed leader election, or idempotent ownership protocol. Calling a local object Singleton does not change system topology.

Prototype copies configured objects

Prototype creates a new object by copying an existing exemplar. It can be useful when setup is expensive or configurations share a complex baseline.

```
final class ReportTemplate {
    private final String title;
    private final List<Section> sections;
    private final Map<String, String> styles;

    ReportTemplate copy() {
        return new ReportTemplate(
            title,
            sections.stream().map(Section::copy).toList(),
            Map.copyOf(styles));
    }
}
```

Java's `Cloneable` and `Object.clone()` provide field-wise copying with awkward contracts, checked exceptions, constructor bypass concerns, and shallow-copy surprises. Copy constructors or named `copy` methods are usually clearer.

```
shallow copy
original ----> mutable list <---- copy

deep/selective copy
original ----> list A
copy      ----> list B
immutable elements may be shared safely
```

Deep copy is not always correct. Identity-bearing entities, connections, locks, threads, and caches may not be meaningfully cloneable. Define which components are copied, shared, reset, or rejected. Immutable values need no defensive deep copy and can be reused safely.

Serialization round-trips are a poor generic cloning strategy: slow, schema-coupled, security-sensitive, and often semantically wrong.

Dependency Injection separates construction from use

Dependency injection supplies collaborators from outside the consuming object. It applies inversion of control: business code declares what it needs, while a composition root decides how to build the object graph.

```
final class CheckoutService {
    private final PaymentGateway gateway;
    private final OrderRepository orders;
    private final DomainEvents events;

    CheckoutService(PaymentGateway gateway,
                    OrderRepository orders,
                    DomainEvents events) {
        this.gateway = Objects.requireNonNull(gateway);
        this.orders = Objects.requireNonNull(orders);
        this.events = Objects.requireNonNull(events);
    }
}
```

```
composition root
|
+-> build/configure gateway
+-> build repository
+-> build event publisher
+-> inject CheckoutService
```

business object uses dependencies; it does not locate them

Constructor injection makes required dependencies visible and supports immutability. Setter injection is appropriate for genuinely optional or reconfigurable collaborators but permits partially configured objects. Field injection hides requirements and complicates plain unit tests.

Dependency injection is not automatically a container. Manual composition is excellent for small systems. A container adds lifecycle, scopes, conditional registration, proxying, configuration, and extension mechanisms. Those features are valuable at scale but make startup behavior and runtime types less obvious.

Dependency Injection versus Service Locator

A service locator lets code request dependencies from a global registry:

```
PaymentGateway gateway = Services.get(PaymentGateway.class);
```

The dependency is not visible in the constructor, and failure moves to runtime lookup. DI pushes resolution to the composition boundary. Some framework APIs necessarily expose registries, but domain services should normally receive explicit dependencies.

A constructor with 15 dependencies may indicate that the class has too many responsibilities. Do not "solve" it with field injection or a locator; reconsider cohesion and orchestration boundaries.

Scope mismatch and lifecycle ownership

Construction patterns must define scope:

- value/request object: new for one operation;
- application service: often one per application context;
- connection pool: shared, bounded, explicitly closed;
- transaction/session: short-lived and context-bound;
- prototype collaborator: new per injection/request;

- tenant resource: keyed and evicted by tenant lifecycle.

Injecting a request-scoped object directly into an application singleton requires a proxy/provider because the actual instance changes per request. Retaining that scoped object beyond its lifecycle is a bug even if injection succeeds.

Factories returning `AutoCloseable` resources should state who closes them. A factory that hides creation must not hide ownership.

Spring as a creational-pattern engine

Spring's `BeanFactory` and `ApplicationContext` act as configurable factories and registries. Bean definitions describe construction, dependency resolution, scope, lifecycle callbacks, and post-processing. `FactoryBean` lets a bean produce another object, while `ObjectProvider` supports optional or delayed resolution.

Spring Boot auto-configuration selects families of infrastructure based on classpath, properties, and missing beans. This resembles factories and abstract factories, but the condition system and container lifecycle are the important concrete mechanisms.

Proxies may mean the injected runtime object is not the implementation class. Construction, initialization, and proxying are separate phases. Calling overridable methods from constructors or expecting transactional behavior during initialization can fail because the proxy boundary is not active yet.

Modern Java alternatives

Records reduce builder and value-object boilerplate. Sealed interfaces make factory switches exhaustive over a controlled hierarchy. Lambdas implement small suppliers and strategies. Pattern matching can make result handling explicit. None of these removes the need to decide ownership, compatibility, validation, and lifecycle.

```
sealed interface StoreConfig permits FileStoreConfig, S3StoreConfig {}

static BlobStore create(StoreConfig config) {
    return switch (config) {
        case FileStoreConfig file -> new FileBlobStore(file.path());
        case S3StoreConfig s3 -> new S3BlobStore(s3.bucket(), s3.client());
    };
}
```

This direct exhaustive factory may be better than a class hierarchy when the variants are closed and owned by one module. Use registration or plugins when variants must be open to third parties.

Configuration is part of the creation contract

In production, a factory rarely chooses an implementation from a hard-coded enum alone. Selection is usually driven by configuration, tenant capability, feature rollout, deployment environment, or data already stored in the system. That makes configuration parsing part of the correctness boundary.

Suppose a payment factory accepts `provider=STRIPE`. The important questions are not only which constructor runs. What happens when the value is absent, misspelled, deprecated, or unsupported in a region? Can configuration change while requests are running? Does a fallback silently change business behavior? A senior design validates configuration at startup when possible and fails with a message that names the invalid value and supported alternatives. Silent fallback is appropriate only when it is an explicit product rule and is observable.

```

external configuration
  |
  v
parse and validate ---- invalid ----> fail startup with context
  |
  v
typed creation policy
  |
  v
factory / composition root ----> constructed object graph

```

Typed configuration keeps raw strings from spreading through the application. Secrets should enter through a dedicated credentials abstraction rather than becoming ordinary builder fields, loggable record values, or exception text. If configuration may reload, decide whether existing instances retain the old policy, are replaced atomically, or consult a live provider. Each model has different concurrency and lifecycle consequences.

Failure atomicity and resource ownership

Construction can fail halfway through. A factory may open a connection pool, create a scheduler, register a metric, and then fail while building the final client. If the earlier resources are not closed, repeated retries produce a startup leak even though no request was served.

Treat multi-step construction like a small transaction: either return a fully valid object whose ownership is clear, or release everything acquired so far. Java's try-with-resources helps when intermediate resources implement [AutoCloseable](#), but ownership transfer must be deliberate.

```

SearchClient create(SearchConfig config) {
    ExecutorService executor = Executors.newFixedThreadPool(config.workers());
    try {
        Transport transport = Transport.connect(config.endpoint());
        return new SearchClient(transport, executor); // client now owns both
    } catch (RuntimeException failure) {
        executor.shutdownNow();
        throw failure;
    }
}

```

The constructor or factory documentation should say whether the returned object owns supplied collaborators. Closing a client that merely borrows a shared executor must not shut down the entire application. Conversely, an internally created executor must not become an invisible non-daemon thread that prevents shutdown. Ownership, lifetime, and close order are part of the API even when the type system cannot express all of them.

Factories at version and compatibility boundaries

Factories are especially valuable when persisted data or messages outlive the current code. A deserializer may read an object created under schema version 2 while the application now writes version 5. Construction then includes migration, defaulting, and compatibility policy.

Keep migration separate from ordinary domain behavior. Parse the external form, validate its version, migrate into a current representation, and only then create the domain object. This gives tests a stable place to verify old fixtures. It also prevents every domain method from carrying historical null checks.

Abstract Factory can express protocol families here: a V1 encoder, decoder, validator, and error mapper belong together. Mixing the V1 encoder with the V2 validator may produce a payload that looks valid locally but fails remotely. The family boundary makes that incompatibility visible.

Performance and observability of creation

Creation policy affects latency and memory. Building an immutable value per request is normally cheap; constructing a TLS client, parsing a large schema, or warming a model may not be. Measure before pooling arbitrary objects. Object pools add contention, stale-state risk, and complex return discipline, so they are justified mainly for resources whose setup is genuinely expensive or externally limited.

Observe important factories with counters and timings for creation success, failure category, selected implementation, and initialization duration. Do not attach high-cardinality values such as customer identifiers to metrics. A slow application startup may otherwise appear as an unexplained deployment timeout. Lazy initialization can shorten startup but moves failure and latency to the first request; eager initialization detects invalid configuration earlier. The correct choice depends on availability and deployment strategy, not on a universal rule.

Choosing the smallest adequate construction technique

The following progression avoids ceremonial abstractions:

```
clear constructor
|
named static factory for meaning or caching
|
simple factory for centralized selection
|
builder for staged optional construction
|
registry/provider for extensibility
|
DI container for a large managed object graph
```

This is not a maturity ladder. A large system should still use plain constructors for most values. Move upward only when a concrete pressure appears. A named static factory such as `Money.zero(currency)` communicates meaning without creating an interface hierarchy. A `switch` over a sealed type can be clearer than reflection. A DI container is valuable at the application boundary but should not become a runtime lookup mechanism inside domain code.

The interview signal is judgment: name the force, select the smallest technique that isolates it, and explain the new failure modes introduced by that choice.

Testing construction boundaries

Test more than returned class names:

- invalid and missing configuration;
- every supported variant and family compatibility;
- default behavior and version migration;
- builder defensive copies and reuse policy;
- singleton concurrent initialization and class-loader scope assumptions;
- prototype deep/shared components;
- DI graph failures and circular dependencies;
- resource closure and failed-construction cleanup;
- provider registration conflicts;
- deterministic selection and telemetry.

Integration tests should start the real composition root with representative configuration. Unit tests can inject small fakes without starting the container.

Common senior-level traps

- Creating a factory for a constructor with no selection policy.
- Calling every static creation method Factory Method.
- Growing an abstract factory into an unrelated service locator.
- Using Builder for tiny immutable records without optional complexity.
- Validating builder fields but forgetting defensive copies.
- Treating a local Singleton as cluster-wide uniqueness.
- Using global Singleton access and hiding dependencies.
- Using `clone()` without defining shallow versus deep ownership.
- Injecting short-lived state into a long-lived object without a provider/proxy.
- Assuming the DI container fixes poor cohesion or circular design.
- Hiding resource ownership behind creation abstractions.
- Using reflection or registration without trust and version controls.

A strong interview answer

I start with a constructor or named static factory and introduce a pattern only when construction contains changing policy. A simple factory centralizes implementation selection; Factory Method varies one creation step inside a stable workflow; Abstract Factory creates a compatible product family; Builder names optional steps and validates a complete immutable result; Prototype copies a defined ownership graph; and Singleton controls instance count but global access often harms testability. Dependency injection moves object-graph composition to an explicit root and separates scope from use. In production I define lifecycle, closure, configuration validation, thread safety, class-loader or container scope, and failure cleanup.

Chapter summary

Creational patterns manage selection, staged construction, copying, scope, and object-graph composition. Their value is the policy boundary they create, not the number of pattern-named classes. Modern Java can express factories with switches, lambdas, records, sealed hierarchies, and service providers. Dependency-injection containers industrialize creation and lifecycle but also introduce scope, proxy, and startup complexity. The senior design goal is explicit ownership with the smallest abstraction that keeps construction change out of business behavior.

Revision Notes

- Start with a constructor or named static factory.
- Introduce a pattern when construction contains changing policy or lifecycle.
- A simple factory centralizes selection.
- Factory Method varies a creation step within a stable workflow.
- Lambdas or suppliers can implement factory behavior through composition.
- Abstract Factory creates compatible families of related products.
- Registries enable extensions but require key, trust, and lifecycle policy.
- Builder names optional construction and validates the complete result.
- Builders must defensively copy mutable inputs.
- Singleton instance scope and global access are separate concerns.
- Enum and holder idioms use safe JVM initialization.
- One instance per class loader/context is not one per cluster.
- Prefer injected application scope over global Singleton access.
- Prototype requires explicit shallow/deep/shared ownership semantics.
- Copy methods are normally clearer than `Cloneable`.
- Constructor injection exposes required dependencies.
- DI moves composition to a root; Service Locator hides dependencies.
- Scope mismatch requires a provider/proxy and lifecycle discipline.
- Creation abstractions must define who closes resources.
- Spring combines factories, registries, scopes, lifecycle, and proxies.

Likely interview question: Factory Method versus Abstract Factory?

Factory Method varies one product-creation operation, often inside a stable creator workflow. Abstract Factory supplies a coherent family of related products so callers do not combine incompatible implementations. Composition can implement either without inheritance.

Likely interview question: When should you use Builder?

Use it for many optional parameters, readable staged construction, evolving defaults, or cross-field validation. Keep required identity explicit, validate at build time, copy mutable data, and avoid it when a small constructor or record is already clear.

Likely interview question: Why is Singleton often criticized?

Global access hides dependencies, couples tests to shared state, and obscures lifecycle. One application-scoped instance may be appropriate, but inject it through an abstraction. A JVM singleton also does not provide distributed uniqueness.

Likely interview question: Dependency Injection versus Factory?

A factory creates or selects an object when requested. DI describes a broader composition approach in which required collaborators are supplied externally, often using factories internally. They are complementary rather than mutually exclusive.